

Git & GitHub Troubleshooting Cheat Sheet

Description

This is a quick-reference troubleshooting guide for the everyday problems people run into while using Git and GitHub — staging and committing changes, pushing/pulling, merge and rebase conflicts, branch mix-ups, remote/authentication errors, and GitHub-specific issues like failing Actions or rejected large files. Each entry explains **what the problem/error looks like**, followed by the **command(s) to fix it**. It's organized so you can either read it top to bottom to learn the workflow, or jump straight to the section matching your error message.

How to Use This Cheat Sheet

1. **Identify your symptom.** Look at the error message Git/GitHub gave you, or describe what went wrong (e.g., "push rejected", "wrong commit message", "merge conflict").
2. **Jump to the matching section** using the table of contents below, or use the **Quick Decision Guide** (Section 11) for a fast symptom → fix lookup.
3. **Run the diagnostic commands first** (Section 10) if you're unsure what state your repo is in — `git status`, `git log --oneline`, and `git reflog` will tell you almost everything you need before you act.
4. **Copy-paste the commands** into your terminal from inside the relevant Git repository folder. Replace placeholders like `<file>`, `<branch>`, `<hash>`, and `<commit-hash>` with your actual values.
5. **Read the warning icons (⚠) carefully** — some commands (force-push, `reset --hard`, history rewrites) permanently discard work or overwrite the remote. When in doubt, create a backup branch first (see the note at the very end of this document).
6. **Keep this file handy** — save it, bookmark it, or keep it open in a split pane while you work so you're not searching Stack Overflow mid-panic.

Table of Contents

1. Basic Workflow Issues
 2. Undoing Things
 3. Push Problems
 4. Merge & Rebase Conflicts
 5. Rewriting History (Advanced / Risky)
 6. Branch Issues
 7. Remote / Repository Issues
 8. GitHub-Specific Issues
 9. Authentication & Access
 10. Diagnostic Commands (Run These First When Stuck)
 11. Quick Decision Guide
-

1. Basic Workflow Issues

Covers: the everyday stage → commit → push cycle, and the small mistakes that happen while doing it (forgetting a file, wrong message, wrong branch, "nothing to commit" confusion).

"Changes not staged for commit"

```
bash

git status          # see what's changed
git add <file>      # stage a specific file
git add .           # stage everything in current dir
git add -A          # stage everything (incl. deletions) repo-wide
```

"Nothing to commit, working tree clean" but you expected changes

- Check you're on the right branch: `git branch`
- Check you're in the right repo folder: `pwd`
- File might be in `.gitignore`: `git check-ignore -v <file>`

Forgot to add a file before committing

```
bash

git add <forgotten-file>
git commit --amend --no-edit
```

Wrote the wrong commit message

```
bash

git commit --amend -m "New correct message"
```

(Only safe if you haven't pushed yet. If pushed, see Section 5 — rewriting pushed history.)

Committed to the wrong branch

```
bash

git log              # note the commit hash
git reset --soft HEAD~1  # undo commit, keep changes staged
git stash            # stash changes
git checkout correct-branch
git stash pop
git commit -m "message"
```

2. Undoing Things

Covers: reversing mistakes — discarding local edits, unstaging, undoing commits (with or without keeping changes), recovering "lost" commits, and cleaning stray untracked files.

Undo uncommitted changes in a file

```
bash

git checkout -- <file>      # old syntax
git restore <file>          # newer syntax
```

Undo all uncommitted changes (careful — destructive)

```
bash

git reset --hard HEAD
```

Unstage a file (keep the changes)

```
bash

git reset <file>
git restore --staged <file>  # newer syntax
```

Undo the last commit but keep changes (unstaged)

```
bash

git reset HEAD~1
```

Undo the last commit but keep changes (staged)

```
bash

git reset --soft HEAD~1
```

Undo the last commit and discard changes completely

```
bash

git reset --hard HEAD~1
```

Undo a commit that's already pushed (safe way — no history rewrite)

```
bash
```

```
git revert <commit-hash>
git push
```

Recover a "lost" commit (after reset --hard, accidental delete, etc.)

```
bash

git reflog                # find the commit hash
git checkout <hash>       # inspect it
git branch recovery-branch <hash> # save it to a new branch
```

Delete an untracked file/folder accidentally left around

```
bash

git clean -n    # dry run - shows what would be deleted
git clean -f    # actually delete untracked files
git clean -fd   # also delete untracked directories
```

3. Push Problems

Covers: rejected pushes, non-fast-forward errors, force-pushing safely, pushing to the wrong branch, and SSH/HTTPS connection failures during push.

"Updates were rejected because the remote contains work that you do not have locally"

```
bash

git pull origin <branch> --rebase # preferred: keeps history linear
# or
git pull origin <branch>          # creates a merge commit
git push origin <branch>
```

"failed to push some refs" (non-fast-forward)

Same as above — pull/rebase first, resolve conflicts if any, then push.

Force-push needed (after amending/rebasing local history)

```
bash

git push --force-with-lease origin <branch> # safer - checks remote hasn't changed
git push --force origin <branch>           # dangerous - overwrites remote updates
```

⚠ Never force-push to a shared branch (main/master) unless the whole team knows.

Pushing to the wrong remote branch

```
bash
git push origin HEAD:correct-branch-name
```

"Permission denied (publickey)"

```
bash
ssh -T git@github.com      # test SSH connection
```

- Check SSH key exists: `ls -al ~/.ssh`
- Generate a new key: `ssh-keygen -t ed25519 -C "your_email@example.com"`
- Add it to the ssh-agent: `ssh-add ~/.ssh/id_ed25519`
- Add the public key to GitHub → Settings → SSH and GPG keys

"remote: Support for password authentication was removed"

GitHub no longer accepts account passwords over HTTPS. Use one of:

- A Personal Access Token (PAT) in place of the password
- SSH instead of HTTPS (switch remote URL: `git remote set-url origin git@github.com:user/repo.git`)
- GitHub CLI auth: `gh auth login`

Push is hanging / very slow

- Check repo size / large files: `git count-objects -vH`
- Check network/proxy settings: `git config --global http.proxy`
- Try SSH instead of HTTPS or vice versa

4. Merge & Rebase Conflicts

Covers: what to do when Git can't auto-merge changes — resolving conflict markers, continuing/aborting a rebase, and choosing between "ours" vs "theirs" versions of a file.

Merge conflict during pull/merge

```
bash
git status                # see conflicted files
# open each file, resolve the <<<<<< ===== >>>>>> markers
git add <resolved-file>
git commit                # completes the merge
```

Merge conflict during rebase

```
bash

# resolve conflicts in each file
git add <resolved-file>
git rebase --continue
# repeat until rebase finishes
# to bail out entirely:
git rebase --abort
```

Abort a merge in progress

```
bash

git merge --abort
```

See both versions of a conflicted file clearly

```
bash

git diff                # shows conflict markers in context
git checkout --ours <file>  # keep your version
git checkout --theirs <file> # keep incoming version
```

Too many conflicts, want to redo the merge from scratch

```
bash

git merge --abort
git status      # confirm clean
# try again, possibly with a different strategy
```

5. Rewriting History (Advanced / Risky)

Covers: squashing commits, interactive rebase, removing sensitive files/secrets from history, and fixing commit author info. These commands rewrite commit hashes — use with caution on shared branches.

Squash last N commits into one

```
bash

git reset --soft HEAD~N
git commit -m "Combined commit message"
git push --force-with-lease
```

Interactive rebase to edit/reorder/squash commits

bash

```
git rebase -i HEAD~N
# in the editor: pick / squash / reword / drop as needed
```

Remove a sensitive file from entire history (e.g., leaked secret)

bash

```
# modern approach using git-filter-repo (recommended over filter-branch)
pip install git-filter-repo
git filter-repo --path path/to/secret-file --invert-paths
git push origin --force --all
```

⚠ After this, rotate/revoke the leaked secret regardless — history rewrite doesn't undo exposure.

Change author name/email on past commits

bash

```
git rebase -i --root
# mark commits as 'edit', then for each:
git commit --amend --author="New Name <new@email.com>" --no-edit
git rebase --continue
```

6. Branch Issues

Covers: recovering deleted branches, diverged branches, blocked deletes, detached HEAD state, and renaming branches locally/remotely.

Deleted a branch by accident

bash

```
git reflog # find the last commit hash on that branch
git branch <branch-name> <hash> # recreate it
```

Local branch and remote branch have diverged

bash

```
git status # tells you how many commits ahead/behind
git pull --rebase origin <branch>
```

Can't delete a branch ("not fully merged")

bash

```
git branch -d branch-name      # safe delete, fails if unmerged
git branch -D branch-name      # force delete
```

Delete a remote branch

bash

```
git push origin --delete branch-name
```

Wrong branch checked out / detached HEAD state

bash

```
git status                      # confirms "HEAD detached at <hash>"
git checkout -b new-branch-name # save work into a proper branch
# or, if you just want to go back:
git checkout main
```

Rename a branch

bash

```
git branch -m old-name new-name
git push origin -u new-name
git push origin --delete old-name
```

7. Remote / Repository Issues

Covers: "not a git repository" errors, wrong/missing remote URLs, clone failures, unrelated-history merge errors, and slow/oversized repos.

"fatal: not a git repository"

bash

```
git init                        # if this should be a new repo
cd correct-folder              # or you're in the wrong directory
```

Wrong remote URL / need to change it

bash

```
git remote -v # check current remotes
git remote set-url origin <new-url>
```

Add a remote that doesn't exist yet

```
bash
git remote add origin <url>
```

Cloning fails / times out

- Check URL is correct (typo, private repo without access)
- Try SSH vs HTTPS
- Check firewall/VPN/proxy interference

"fatal: refusing to merge unrelated histories"

```
bash
git pull origin main --allow-unrelated-histories
```

(Happens when local and remote repos were initialized separately.)

Repository too large / slow clone

```
bash
git clone --depth 1 <url> # shallow clone, only latest commit
```

8. GitHub-Specific Issues

Covers: problems specific to the GitHub platform — PR conflicts, wrong PR base branch, failing Actions workflows, files rejected for being too large, and syncing a fork with upstream.

Pull Request shows conflicts

```
bash
git checkout main
git pull origin main
git checkout your-branch
git merge main # or: git rebase main
# resolve conflicts, then
git push
```

Accidentally opened PR against wrong base branch

- On GitHub: open the PR → click "Edit" near the title → change base branch dropdown.

GitHub Actions failing after push

- Check the Actions tab logs for the specific failing step
- Common causes: missing secrets/env vars, wrong Node/Python version, failing tests, YAML syntax errors in `.github/workflows/*.yaml`

Large file rejected by GitHub (>100MB)

```
bash

git rm --cached large-file
echo "large-file" >> .gitignore
git commit -m "Remove large file"
```

For legitimately large files, use Git LFS:

```
bash

git lfs install
git lfs track "*.psd"
git add .gitattributes
```

"This branch has conflicts that must be resolved" on GitHub UI

Resolve locally (Section 4) and push — GitHub UI conflict editor works too for simple cases, but local resolution is safer for complex conflicts.

Fork is behind the upstream repo

```
bash

git remote add upstream <original-repo-url>
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

9. Authentication & Access

Covers: expired tokens, "repository not found" errors, credential caching, and managing multiple GitHub accounts on one machine via SSH config.

Token expired / invalid credentials

- Regenerate PAT: GitHub → Settings → Developer settings → Personal access tokens

- Update stored credentials:

```
bash
```

```
git config --global credential.helper store # or 'cache' for temporary
```

On macOS, clear Keychain entry for github.com if it's caching an old token.

"Repository not found" (but it exists)

- You likely don't have access — check org/repo permissions
- Or it's a typo in the URL
- Or you're authenticated as the wrong GitHub account: `gh auth status`

Multiple GitHub accounts on one machine

```
bash
```

```
# use SSH config to separate identities
# ~/.ssh/config
Host github.com-work
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_work

Host github.com-personal
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_personal
```

Then clone using: `git clone git@github.com-work:org/repo.git`

10. Diagnostic Commands (Run These First When Stuck)

Covers: the "look before you leap" commands — run these to understand your repo's current state before running any fix above, especially destructive ones.

```
bash
```

```

git status                # current state, staged/unstaged files
git log --oneline -10     # recent commit history
git branch -vv            # branches + tracking info
git remote -v             # remote URLs
git diff                  # unstaged changes
git diff --staged         # staged changes
git reflog                # full history of HEAD movements (lifesaver for recovery)
git fsck --lost-found     # find dangling/orphaned commits

```

11. Quick Decision Guide

Covers: a fast lookup table — match your symptom in the left column to the fix in the right column.

Symptom	Likely Fix
Push rejected	<code>git pull --rebase</code> then push
Wrong commit message (not pushed)	<code>git commit --amend</code>
Wrong commit message (pushed)	amend + <code>push --force-with-lease</code>
Accidentally committed to wrong branch	<code>git reset --soft HEAD~1</code> + stash + move
Need to undo a pushed commit safely	<code>git revert</code>
Merge conflict	resolve markers → <code>add</code> → <code>commit</code> <code>rebase --continue</code>
Lost commits after reset	<code>git reflog</code> → recover hash
Detached HEAD	<code>git checkout -b new-branch</code>
Can't authenticate	check PAT/SSH key setup
Large file rejected	remove + <code>.gitignore</code> or use Git LFS

General safety rule: before any destructive command (`reset --hard`, `push --force`, `clean -f`, history rewrites), make sure you actually need it — `git reflog` can often save you, but not always. When in doubt, create a backup branch first:

```
bash
```

```
git branch backup-before-risky-operation
```